RECIPES API DOKUMENTATSIOON

1. Sissejuhatus

Käesolev dokumentatsioon tutvustab API, mis kasutab **Clerk** autentimist ja **MongoDB** andmebaasi. Rakendus võimaldab kasutajatel retsepte (postitusi) luua, kommenteerida ning salvestada oma lemmikpostitusi. Lisaks on toetatud kasutajate registreerimine, sisselogimine ja haldamine, samuti postituste ja kommentaaride kustutamine.

Põhifunktsioonid:

- **Autentimine ja autoriseerimine:** Kasutatakse Clerk'i teenust, et hallata kasutajate registreerimist ja sisselogimist. Kasutajate andmed salvestatakse **MongoDB** andmebaasi.
- **Postituste haldamine:** Kasutajad saavad luua uusi retsepte, vaadata olemasolevaid ja kustutada oma retsepte.
- **Kommentaaride lisamine:** Kasutajad saavad postitustes kommentaare lisada, lugeda ja kustutada.
- **Salvestamine:** Kasutajad saavad salvestada oma lemmikpostitused ja hiljem neid vaadata.
- **Admini tugi:** Administraatoritel on ligipääs postituste ja kommentaaride haldamiseks kogu rakenduses. Administraator saab postitusi esile tõsta (kuvatakse esilehel).

Arhitektuur ja tehnoloogiad:

- **Backend:** Rakendus kasutab **Node.js** koos **Express** raamistikuga API loomiseks. Autentimine toimub **Clerk** teenuse kaudu.
- **Andmebaas:** Kasutatakse **MongoDB** andmebaasi, kus andmed salvestatakse **Mongoose** abil.
- **Frontend:** Rakendus kasutab **React (Vite)** raamistiku abil kasutajaliidese loomiseks. Kasutajad saavad sisse logida, postitusi lugeda ja kommenteerida. **Hetkel veel arenduses ja loodud ainult backend testimise eesmärgil, et kontrollida, kas Clerk töötab ja kas andmebaasiga on ühendatud.**

Link GitHub repositooriumile: https://github.com/vitalitrifonov/recipes-api

Link Backend videole: https://youtu.be/7zfhRy9kGaM

Link Frontend videole: https://youtu.be/tIg-GL_jKtc

2. Kuidas käivitada API ja Frontend

Järgnevalt leiate sammud, kuidas käivitada rakenduse **API** ja **Frontend**:

- **Kuidas käivitada Backend (API)**

API käivitamiseks tuleb järgida järgmisi samme:

Eeltingimused:

- Node.js: Veenduge, et teil on arvutis paigaldatud Node.js (soovitatavalt versioon 16+).
- MongoDB: Veenduge, et teil on juurdepääs MongoDB andmebaasile.
- .env fail: Kõik keskkonfiguratsioonid ja tundlikud andmed (nt andmebaasi URL, Clerk'i võtmed jne) on salvestatud .env failis.

Käivitusprotsess:

Klooni projekti: Kui projekt ei ole veel kloonitud, tehke seda järgmiselt:

```
git clone <projektide repo URL>

cd <projekti kaust>
```

Paigaldage sõltuvused: Veenduge, et kõik vajalikud Node.js sõltuvused on paigaldatud. Seda saab teha järgmise käsuga:

```
npm install
```

.env faili seadistamine: Veenduge, et teil oleks olemas .env fail, kus on kõik vajalikud keskkonfiguratsioonid. Kui see fail puudub, looge see ja lisage oma keskkonfiguratsioonid.

Käivitage API server: API serveri käivitamiseks kasutage järgmist käsku:

```
node --watch --env-file .env index.js
```

- **--watch**: See käivitab API automaatselt ümber, kui faili muudatused on tehtud.
- **--env-file .env**: See võimaldab dotenv moodulil lugeda keskkonnamuutujad .env failist.

Kui kõik on õigesti seadistatud, peaks server käivituma ja kuulatama porti **3000**.

Testige API serverit: Avage oma brauser ja külastage järgmist URL-i:

```
http://localhost:3000
```

Testimine ngrok abil

Kasutatakse Clerk kasutaja registreerimise/sisselogimise testimiseks, kuna Clerk ei toeta localhost'i.

Installige ngrok: Kui teil pole veel ngroki installitud, saate seda teha Administrator Command Prompt'is Windows platvormil järgmiselt:

```
choco install ngrok
```

Ühendage oma kasutajaga:

```
ngrok config add-authtoken <TOKEN>
```

Käivitage ngrok: Käivitage **ngrok**, et luua ajutine avalik URL teie localhost'ile:

```
ngrok http http://localhost:3000
```

Lisage ajutine URL (nt https://875f-80-235-95-30.ngrok-free.app) Clerk webhook endpointi.

Frontendi käivitamine

Selle projekti frontend on üles ehitatud React + Vite stackil. Allpool leiad samm-sammulise juhendi arenduskeskkonna käivitamiseks.

Veendu, et sul oleks installitud:

- Node.js (soovitatav versioon 18.x või uuem)
- npm (tuleb Node'iga kaasa)
- Clerk konto ja .env seadistused (vajalik autentimiseks)

Installimine

Liigu projekti frontend kausta.

```
cd public
```

Paigalda vajalikud sõltuvused:

```
npm install
```

Frontendi käivitamine arendusrežiimis

Kui kõik sõltuvused on paigaldatud, käivita arenduskeskkond käsuga:

```
npm run dev
```

Vite käivitab automaatselt lokaalserveri (nt http://localhost:5173) ja uuendab lehte reaalajas, kui teed muudatusi koodis.

3. Autentimine ja autoriseerimine

API kasutab Clerk'i autentimist, mille kaudu saab mõned API funktsioonid kasutada ainult sisse logitud kasutajad. Kõik päringud, mis nõuavad autentimist, kontrollivad, kas kasutajal on kehtiv sessioon.

Clerk autentimise seadistamine:

- Kui kasutaja ei ole sisse logitud, ei pääse ta piiratud API lõpp-punktidesse.
- API kasutab clerkMiddleware ja requireAuth filtreid.

Ülevaade Clerk'ist

Clerk on pilvepõhine autentimise teenus, mis pakub lihtsat kasutajate haldust, sealhulgas registreerimist, sisselogimist, paroole, sotsiaalset autentimist ja kaheastmelist autentimist (2FA). Clerk on integreeritud React-i, Node.js-i ja teiste tehnoloogiatega, et pakkuda turvalist ja lihtsat autentimist. Rakenduses on Clerk kasutusel nii frontendis (React) kui ka backendis (Node.js).

Frontend (React) - Kasutaja Registreerimine ja Sisselogimine

Frontend on veel arenduses ja loodud ainult backend'i testimise eesmärgil, et kontrollida, kas Clerk töötab ja kas andmebaasiga on ühendatud.

App.jsx - Registreerimise ja Sisselogimise Komponendid:

- **SignIn** ja **SignUp** komponendid: Need on Clerk'i paketi valmis loodud komponendid, mida saab kasutada sisselogimiseks ja registreerimiseks.
- **UserButton**: Kui kasutaja on sisse logitud, kuvatakse nupp, mis võimaldab tal välja logida. Pärast väljalogimist suunatakse kasutaja **SignUp** lehele.
- **useUser** hook: Kasutatakse kasutaja oleku kontrollimiseks (kas kasutaja on sisse logitud või mitte).
- **handleGetToken**: Kui kasutaja on sisse logitud, saab ta kasutada nuppu, et hankida **JWT (JSON Web Token)**, et testida kasutaja võimalusi nt Postmanis.

Backend (Node.js) - Autentimine ja Kasutajate Hallatavus

Backendis kasutab rakendus **Clerk** API-d kasutajate autentimiseks ja haldamiseks. Clerk-i autentimine toimub järgmiselt:

- **Clerk Middleware (clerkMiddleware)**: Backendis toimub kasutaja autentimine ja autoriseerimine **Clerk**-i **middleware** abil. See middleware kontrollib iga päringu päist, et kontrollida, kas kasutaja on autentitud. **Päringud, mis ei ole autentitud**: Kui kasutaja ei ole sisse logitud või tema sessioon on lõppenud, siis Clerk tagastab veateate ja päringut ei töödelda.
- **Kasutaja autentimise andmed (req.auth)**: Pärast autentimist on saadaval objekt **req.auth**, mis sisaldab kõiki kasutaja autentimise andmeid, sealhulgas kasutaja ID-d ja muid metaandmeid.

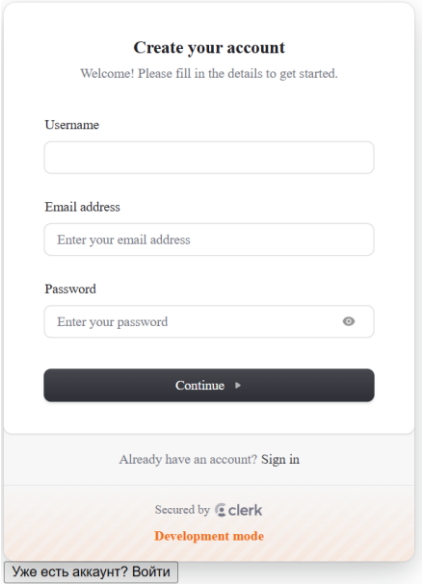
— **Webhooki töötlemine:** Kui kasutaja loob või kustutab oma konto **Clerk** teenuses, saadab **Clerk** Webhook'i serverile, mis käivitab spetsiaalse lõpp-punkti, et täiendavalt töötleda kasutaja andmeid (nt salvestada uus kasutaja andmebaasi või kustutada vanad postitused, kui kasutaja kustutab oma konto). Webhooki kontrollimiseks kasutatakse **svix** teeki, et veenduda, et päring pärineb õigesti autentitud **Clerk** teenuselt.

Kuidas kasutada tokenit ja teha API päringuid:

Token-põhise autentimise korral tuleb iga API päringu jaoks lisada Authorization päis koos kehtiva JWT tokeniga. Autoriseerimise päis sisaldab tokenit järgmises vormingus:

```
Authorization: Bearer <JWT-token>
```

Registreerimise aken kasutades **SignUp** Clerk komponendi:



Pärast andmete sisestamist ja eduka registreerimist kasutaja andmed on kättesaadavad loodud MongoDB andmebaasis:


```
_id: ObjectId('67effa3ecffdc147cf874856')
clerkUserId : "user_2vGnjTb1k9X71Ic4xATAk3fxssi"
username : "test"
email : "test@test.com"
▸ savedPosts : Array (empty)
createdAt : 2025-04-04T15:26:54.769+00:00
updatedAt : 2025-04-04T15:26:54.769+00:00
__v : 0
```


Samuti saab hallata ja luua kasutajaid otse Clerk juhtpaneeli kaudu.

4. Retseptide lisamine API kaudu

Retsepti lisamiseks kasutatakse **POST /posts** endpoint'i, mis võimaldab kasutajatel lisada uue retsepti (postituse) andmebaasi. Endpointi kaitstakse autentimise ja autoriseerimisega, mis tähendab, et kasutaja peab olema sisse logitud ja omama kasutaja õigusi.

Endpoint:

POST /posts

- **Autentimine**: Jah, nõuab JWT tokenit Authorization päises
- **Autoriseerimine**: Jah, ainult registreeritud kasutajad saavad raamatuid lisada
- **Päringu sisu**: JSON andmed, mis sisaldavad retsepti pealkiri, URL-sõbralik versioon pealkirjast (slug), kategooria ja sisu.
- **Vastus**: Eduka lisamise korral tagastatakse loodud retsepti andmed.

Retsepti lisamise päringu (Postman POST) näidis

http://localhost:3000/posts

Retsepti lisamiseks tuleb API-le saata **POST-päring** järgmiste andmetega JSON-formaadis:

```
{
    "title":"Title from Postman-6",
    "slug":"slug-from-postman-6",
    "content":"Content from Postman"
}
```

Postman vastus:

```
    {

    "user": "67effa3ecffdc147cf874856",
    "title": "Title from Postman-6",
    "slug": "slug-from-postman-6",
    "category": "Uncategorized",
    "content": "Content from Postman",
    "isFeatured": false,
    "visit": 0,
    "_id": "6806751327037a5de829643d",
    "createdAt": "2025-04-21T16:40:51.583Z",
    "updatedAt": "2025-04-21T16:40:51.583Z",
    "__v": 0
}
```

Salvestatud retsepti andmed MongoDB andmebaasis:

```
_id: ObjectId('6806751327037a5de829643d')
user : ObjectId('67effa3ecffdc147cf874856')
title : "Title from Postman-6"
slug : "slug-from-postman-6"
category : "Uncategorized"
content : "Content from Postman"
isFeatured : false
visit : 0
createdAt : 2025-04-21T16:40:51.583+00:00
updatedAt : 2025-04-21T16:40:51.583+00:00
__v : 0
```

5. Retseptide vaatamine API kaudu

Retsepti vaatamiseks kasutatakse **GET /posts** endpoint'i, mis võimaldab kasutajatel vaatada kõiki retsepte (postitusi) andmebaasist. Endpoint ei ole autentimise ega autoriseerimisega kaitstud, mis tähendab, et kõik rakenduse külastajad saavad kasutajate retsepte vaadata.

Endpoint:

GET /posts

- **Autentimine**: Ei
- **Autoriseerimine**: Ei
- **Päringu sisu**: JSON andmed, mis sisaldavad kõikide retseptide MongoDB poolt genereeritud unikaalne retsepti ID, viide kasutaja _id-le, pealkiri, URL-sõbralik versioon pealkirjast (slug), kategooria, sisu, kas postitus on esile tõstetud, külastuste arv, loomise aja ja muutmise aja.
- **Vastus**: Eduka päringu korral tagastatakse kõikide loodud retseptide andmed

Retseptide vaatamise päringu (Postman GET) näidis

http://localhost:3000/posts

Retseptide saamiseks tuleb teha GET-päring ilma lisasisuta.

Kui päring on edukas, tagastab server JSON-andmed, mis sisaldavad kõiki retsepte koos seotud andmetega.

Vastuse näidis Postmanis:


```
    {
        "_id": "67ed722a36a9a5f877eb1f5c",
        "user": "67ed64cfc08b20ee53cb862b",
        "title": "Title from Postman-4",
        "slug": "slug-from-postman-4",
        "category": "Uncategorized",
        "content": "Content from Postman",
        "isFeatured": false,
        "visit": 0,
        "createdAt": "2025-04-02T17:21:46.908Z",
        "updatedAt": "2025-04-02T17:21:46.908Z",
        "__v": 0
    },
    {
        "_id": "67eebfa49ffd281a88c955c5",
        "user": "67eebf8b9ffd281a88c955bf",
        "title": "Title from Postman-2",
        "slug": "slug-from-postman-2",
        "category": "Uncategorized",
        "content": "Content from Postman",
        "isFeatured": false,
        "visit": 0,
        "createdAt": "2025-04-03T17:04:36.540Z",
        "updatedAt": "2025-04-03T17:04:36.540Z",
        "__v": 0
    },
    {
        "_id": "6806751327037a5de829643d",
        "user": "67effa3ecffdc147cf874856",
        "title": "Title from Postman-6",
        "slug": "slug-from-postman-6",
        "category": "Uncategorized",
        "content": "Content from Postman",
        "isFeatured": false,
        "visit": 0,
        "createdAt": "2025-04-21T16:40:51.583Z",
        "updatedAt": "2025-04-21T16:40:51.583Z",
        "__v": 0
    }
```

```
]
```

6. Retsepti slug järgi saamine API kaudu

Retsepti vaatamiseks kasutatakse GET /posts endpoint'i, mis võimaldab kasutajatel vaadata retsepte (postitusi) slug järgi andmebaasist. Endpoint ei ole autentimise ega autoriseerimisega kaitstud, mis tähendab, et kõik rakenduse külastajad saavad kasutajate retsepte slug järgi vaadata.

Endpoint:

GET /posts

- **Autentimine**: Ei
- **Autoriseerimine**: Ei
- **Päringu sisu**: Peab sisaldama retsepti slug-i URL-is
- **Vastus**: Tagastab retsepti andmed koos seotud informatsiooniga

Retsepti saamiseks tuleb teha GET-päring ning määrata retsepti slug URL-is.

http://localhost:3000/posts/slug-from-postman-6 - näidis, vaatame „slug-from-postman-6" retsepti andmed.

Kui päring on edukas, tagastab server JSON-andmed, mis sisaldavad retsepti andmeid koos seotud elementidega.

Vastuse näidis Postmanis:


```
{
    "_id": "6806751327037a5de829643d",
    "user": "67effa3ecffdc147cf874856",
    "title": "Title from Postman-6",
    "slug": "slug-from-postman-6",
    "category": "Uncategorized",
    "content": "Content from Postman",
    "isFeatured": false,
    "visit": 0,
    "createdAt": "2025-04-21T16:40:51.583Z",
    "updatedAt": "2025-04-21T16:40:51.583Z",
    "__v": 0
}
```


7. Retsepti kustutamine

Retsepti kustutamiseks kasutatakse **DELETE /posts/:id** endpoint'i. See lubab kasutajatel (ainult oma retsepte) ja administraatoritel (kõiki retsepte) eemaldada koos kõigi seotud andmetega, nagu **kommentaarid ja autorite seosed**. Endpoint on **autentimise ja autoriseerimisega kaitstud**, seega peab kasutaja olema **sisse logitud ja omama admini õigusi**. Kasutajad (ilma admini õiguseta) saavad kustutada ainult oma retsepte.

Endpoint:

DELETE /posts/:id

- **Autentimine**: Jah, nõuab JWT tokenit Authorization päises
- **Autoriseerimine**: Jah, ainult admin või kasutaja, kes on loonud retsepti saab seda kustutada

- **Päringu sisu**: Peab sisaldama retsepti ID-d URL-is
- **Vastus**: Kinnitus edukast kustutamisest või veateade

Retsepti kustutamiseks tuleb teha **DELETE-päring** ning määrata **retsepti ID** URL-is.

http://localhost:3000/posts/6806751327037a5de829643d – näidis kustutame id: 6806751327037a5de829643d retsepti

Kui päring on edukas, tagastab server JSON-andmed kustutamise kinnitusega.

```
"Post has been deleted"
```

8. Retseptile kommentaari lisamine

Raamatu kommenteerimiseks kasutatakse POST /comments/:postId endpoint'i. Selle kaudu saavad autentitud kasutajad lisada kommentaare kindlale raamatule.

Endpoint:

POST /comments/:bookId

- **Autentimine**: Jah, nõuab JWT tokenit Authorization päises
- **Autoriseerimine**: Jah, ainult sisse logitud kasutajad saavad kommentaare lisada
- **Päringu sisu**: Kommenteeritava retsepti ID URL-is ning kommentaari sisu päringukehas
- **Vastus**: Uue kommentaari andmed või veateade

Kommentaari lisamiseks tuleb teha POST-päring ning määrata retsepti ID URL-is.

http://localhost:3000/comments/67eebfa49ffd281a88c955c5 - näidis, lisame kommentaari id: 67eebfa49ffd281a88c955c5 retseptile

```
{
  "desc": "Väga hea retsept"
}
```

Kui päring on edukas, tagastab server **JSON-andmed uue kommentaariga**.

```
{
    "user": "67effa3ecffdc147cf874856",
    "post": "67eebfa49ffd281a88c955c5",
    "desc": "Väga hea retsept",
    "_id": "6807bde56e66b638495b4a41",
    "createdAt": "2025-04-22T16:03:49.917Z",
    "updatedAt": "2025-04-22T16:03:49.917Z",
    "__v": 0
}
```

Salvestatud kommentaari andmed MongoDB andmebaasis:

```
_id: ObjectId('6807bde56e66b638495b4a41')
user : ObjectId('67effa3ecffdc147cf874856')
post : ObjectId('67eebfa49ffd281a88c955c5')
desc : "Väga hea retsept"
createdAt : 2025-04-22T16:03:49.917+00:00
updatedAt : 2025-04-22T16:03:49.917+00:00
__v : 0
```

9. Retsepti kommentaari eemaldamine

Kasutajad saavad kustutada **ainult omaenda kommentaare**. Samuti kommentaari eemaldamise võimalus on adminil. Selleks kasutatakse **DELETE /comments/:commentId** endpoint'i, mis nõuab **autentimist JWT tokeniga**.

Endpoint:

DELETE /comments/:commentId

- **Autentimine**: Jah, nõuab JWT tokenit Authorization päises
- **Autoriseerimine**: Jah, kasutaja saab kustutada ainult enda kirjutatud kommentaare
- **Päringu sisu**: Kommentaari ID URL-is
- **Vastus**: Kustutamise kinnitus või veateade

Kommentaari kustutamiseks tuleb teha **DELETE-päring** ning määrata **kommentaari ID** URL-is.

http://localhost:3000/comments/6807bf433a6a87040901b67f - näidis, eemaldame id: 6807bf433a6a87040901b67f kommentaari

Kui päring on edukas, tagastab server JSON-andmed kustutamise kinnitusega.

```
"Comment deleted"
```

10. Retsepti kommentaaride vaatamine

Retsepti kommentaaride vaatamiseks kasutatakse **GET /comments/:postId** endpoint'i, mis võimaldab kõikidel vaadata kõiki retsepte kommentaarid andmebaasist. Endpoint ei ole autentimise ega autoriseerimisega kaitstud, mis tähendab, et kõik rakenduse külastajad saavad kasutajate retsepte kommentaare vaadata.

Endpoint:

GET /comments/:postId

- **Autentimine**: Ei
- **Autoriseerimine**: Ei
- **Päringu sisu**: Peab sisaldama retsepti ID-d URL-is
- **Vastus**: Eduka päringu korral tagastatakse retsepti kommentaaride andmed

Retsepti kommentaride vaatamise päringu (Postman GET) näidis

http://localhost:3000/comments/67eebfa49ffd281a88c955c5

Retsepti kommentaaride saamiseks tuleb teha GET-päring ilma lisasisuta.

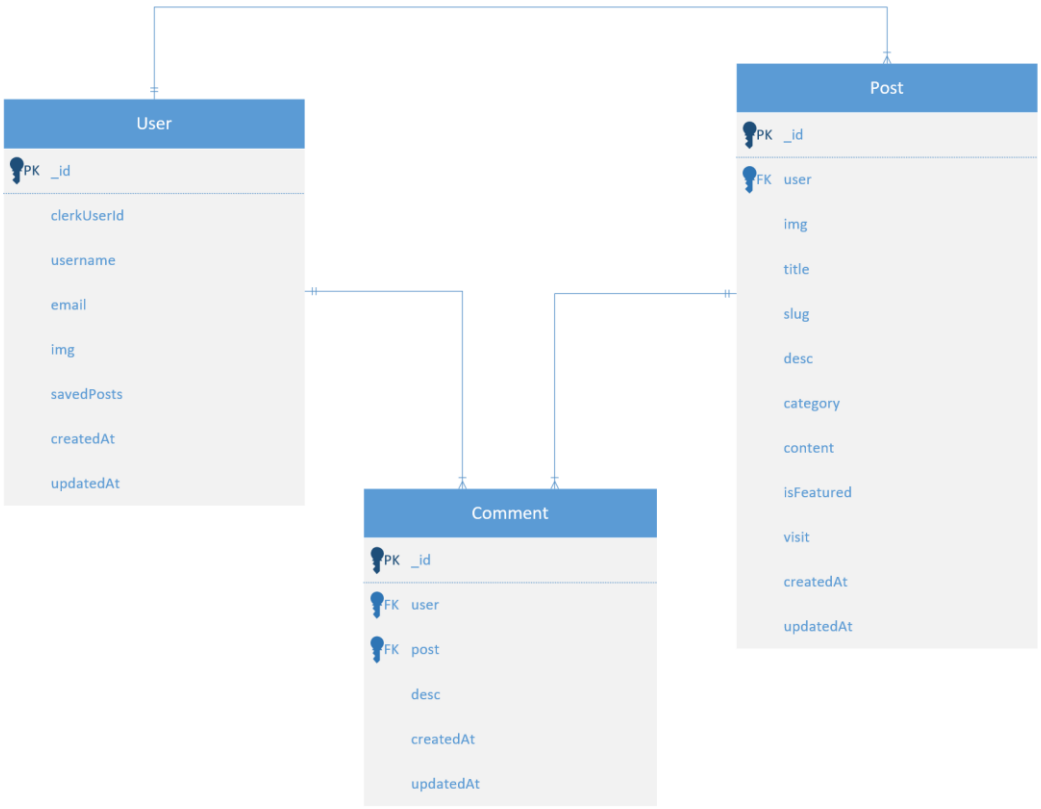
Kui päring on edukas, tagastab server JSON-andmed, mis sisaldavad retsepti kommentaarid koos seotud andmetega.

Vastuse näidis Postmanis:


```
[
    {
        "_id": "6807bde56e66b638495b4a41",
        "user": {
            "_id": "67effa3ecffdc147cf874856",
            "username": "test"
```

```
        },
        "post": "67eebfa49ffd281a88c955c5",
        "desc": "Väga hea retsept",
        "createdAt": "2025-04-22T16:03:49.917Z",
        "updatedAt": "2025-04-22T16:03:49.917Z",
        "__v": 0
    }
]
```


11. Andmebaasi olemi-suhte diagramm



12. Frontend arendamine

Frontend arendus keskendus täielikult funktsionaalse ja kasutajasõbraliku retseptiplatvormi loomisele, mis võimaldab kasutajatel sirvida, otsida, filtreerida, kommenteerida ja lisada uusi retsepte. Arendus hõlmas järgmisi komponente ja lehti:

- **Kasutaja autentimine ja registreerimine:** Integreeriti Clerk autentimissüsteem, mis võimaldab turvalist sisse- ja väljalogimist ning kasutajapõhist sisu haldust.
- **Postituste nimekiri (PostListPage):** Kuvab kõik retseptid koos filtreerimis- ja otsimisvõimalustega.
- **Üksikpostituse leht (SinglePostPage):** Kuvab retsepti detailid, pildi, autori, kategooria ning võimaldab kommenteerida.
- **Uue retsepti lisamine (Write):** Kasutajaliides, mis sisaldab pildi üleslaadimist ja WYSIWYG-redaktorit (ReactQuill) sisutekstide loomiseks ja piltide lisamiseks.
- **Navigatsioon ja layout:** Loodud universaalne paigutus koos navigeerimisribaga (MainLayout) ja marsruutide süsteemiga (React Router).
- **Reaktiivne disain:** Rakendus on täielikult mobiilisõbralik tänu Tailwind CSS-i abil loodud paindlikule kujundusele.
- **Andmepäringud:** Andmete toomine ja uuendamine toimub React Query ja Axios abil, mis võimaldab efektiivset andmehaldust ja puhverdamist.
- **Salvestatud postitused ja otsing:** Eraldi komponent SavedPostList, mis näitab kasutaja salvestatud retsepte.

Väljakutsed ja lahendused

- **Clerk autentimise integreerimine:**

  Väljakutse oli Clerkiga õigete kasutajaandmete hankimine ning tokenite korrektne edastamine backendile. Lahenduseks sai useAuth ja useUser hookide korrektne kasutamine ning turvaline tokeni lisamine Axiosi päringutesse.

- **WYSIWYG-redaktori (React Quill) ja piltide dünaamiline lisamine:**

  Tuli lahendada probleem, kuidas pilte sisestada redaktorisse kasutajasõbralikult. Lahenduseks sai üleslaetud piltide automaatne lisamine setValue() kaudu useEffect hookis.

- **Responsive disain ja komponentide paigutus:**

  Eri ekraanisuurustele sobiva disaini loomine võttis aega. Tailwind CSS-i kasutamine võimaldas siiski kiiret prototüüpimist ja mugavat kujunduse kohandamist.

- **Serveriandmete käsitlemine asünkroonselt:**

  React Query kasutuselevõtt lihtsustas andmete laadimist ja cache'i haldust, ent nõudis õiget struktuuri useQuery ja useMutation hookide ülesehitamisel.

- **Marsruutide haldus ja layouti kontekstid:**

  Marsruutide loomine createBrowserRouter abil võimaldas loogilist lehejaotust, kuid nõudis täpset komponentide ja marsruutide organiseerimist (sh pealispõhine layout).Marsruutide loomine createBrowserRouter abil võimaldas loogilist lehejaotust, kuid nõudis täpset komponentide ja marsruutide organiseerimist (sh pealispõhine layout).

Frontend'is kasutatud tehnoloogiad

Tehnoloogia / Teek	Kirjeldus
Tailwind CSS v3	Utility-first CSS-raamistik, mis võimaldab kiiret ja paindlikku kujunduse loomist otse HTML-klassides. Paigaldatud käsuga npm install -D tailwindcss@3 postcss autoprefixer. Seejärel npx tailwindcss init.
Tailwind Hamburgers	Lihtne hamburger-menüü animatsioonide kogu Tailwind CSS jaoks, kasutatakse mobiilsete navigeerimismenüüde loomiseks. (installimine: npm install tailwind-hamburgers -S)
ImageKit (imagekitio-react)	Pilveteenus piltide üleslaadimiseks, haldamiseks ja optimeerimiseks. Reacti komponentide kaudu võimaldab piltide mugavat integreerimist ja eelvaadet. (installimine: npm install @imagekit/react)
React Router (react-router)	Marsruutide haldusraamistik, mis võimaldab mitmelehelisi (multiple pages) rakenduste loomist Reactis. (installimine: npm i react-router)
Clerk	Kasutajahaldus- ja autentimisplatvorm, mis võimaldab lihtsat sisselogimist, registreerimist ja turvalist ligipääsuhaldust.
ReactQuillNew	Tekstiredaktor (WYSIWYG), mis võimaldab kasutajal postitusi koostada visuaalses vormis – kasutada vormindust, lisada linke, pilte või isegi videoid.
React Query	Andmete pärimiseks, sünkroonimiseks, vahemällu salvestamiseks ja automaatseks uuendamiseks mõeldud tööriist. Aitab vältida boilerplate-koodi ja parandab jõudlust.
Axios	Lubab lihtsasti teha HTTP-päringuid (GET, POST, PUT, DELETE jne) ning suhelda serveri API-ga.
CORS (Cross-Origin Resource Sharing)	Brauseripoolne turvamehhanism, mis määrab, millised domeenid tohivad API-le ligi pääseda. Eriti oluline backendiga suhtlemisel.
React Toastify	Visuaalne teegikogu teavituste (toast) kuvamiseks. Näiteks edukate tegevuste või veateadete näitamiseks.
React Infinite Scroll Component	Võimaldab luua lõputu kerimise funktsionaalsust – andmeid laetakse dünaamiliselt juurde, kui kasutaja jõuab lehe lõppu.
timeago.js	Väike ja tõhus teek kuupäevade „inimkeelses" vormis kuvamiseks (nt „2 minutit tagasi", „3 päeva eest" jne).